Architecture Spec: AI Tabular Data Analyzer

PARADIGM: PLANNER-EXECUTOR | **DOMAIN:** OPERATIONS / INFRA (AGNOSTIC) | **TARGET:** NVIDIA PORTFOLIO

> **THE PRIME DIRECTIVE**
>
> The AI never touches raw numbers. Python (pandas) cleans the data and executes all calculations. The AI only writes plans and turns the finished aggregates into narrative sentences. Letting the AI read raw CSVs leads to hallucinations and hallucinated calculations.

1. DATA QUALITY ENGINE (MESSY DATA HANDLING)

Real-world tabular data is dirty. The pipeline checks the file on upload before the AI is invoked.

PROBLEM	EXAMPLE	SYSTEM ACTION
Invalid File	PDF uploaded, or empty CSV	Halt. Prompt: "Cannot read file. Please upload a CSV."
Missing Target	No identifiable cost/usage column	Halt. Specify which core dimensions are missing.
Empty Cells	Blank cell in usage column	Exclude row from totals; count as "missing value". Do not interpolate.
Type Mismatch	"abc" in a float column	Mark row as invalid; exclude from calculations.
Impossible Values	Negative hours, >100% utilization	Mark row as invalid; exclude from calculations.
Duplicates	Identical exact rows	Deduplicate; record count of duplicates removed.
Missing Periods	No data for a specific date	Do not pad with zeros. Flag dataset as "incomplete period".

Visibility & LLM Context: The UI displays a Data Quality block (e.g., *"1,240 rows · 18 missing · 12 invalid · 1 day missing"*). This exact summary is injected into the LLM context prompt: *"Only use the numbers provided. If data is missing for a period, state it explicitly. Do not estimate."*

2. SMART COLUMN MAPPING

The system does not force rigid schemas. It dynamically maps arbitrary user column names to internal canonical variables using a three-layer approach:

1. **Synonym & Fuzzy Matching (Instant):** Checks against known dictionaries (e.g., `cost` ← price, spend, bill) and fuzzy spelling checks (e.g., `GPU_Hrs`).
2. **AI Semantic Mapping:** If unresolved, the AI receives headers + 5 sample rows and returns a JSON map with confidence scores (e.g., identifies "Monto" as amount/cost).
3. **Code Validation:** Pandas verifies the AI's map (e.g., if AI tags "Monto" as cost, Pandas ensures the column contains numerics).

UI: User Confirmation Step

USER COLUMN (CSV)	MAPPED INTERNAL USE	CONFIDENCE	ACTION
`Price`	Cost	✓ HIGH	Auto-applied
`Client Name`	Partner / Entity	✓ HIGH	Auto-applied
`Hrs`	Usage Metric	⚠ CHECK	Dropdown confirm

3. THE PLANNER-EXECUTOR ARCHITECTURE

The system operates identically to an enterprise agent (like Hisaab AI). The LLM determines *what* to calculate, while Pandas determines *how* to calculate it.

1. **Profile:** Pandas builds a lightweight statistical summary of the dataset (names, types, min/max, null counts).
2. **Plan:** The AI receives the profile and generates an analytical plan in strict JSON.
3. **Validate:** Python verifies the JSON against a strict whitelist of ~15 safe operations (sum, mean, count, top_n, week_over_week).
4. **Execute:** Pandas runs the operations and stores the aggregate results.
5. **Report:** The AI receives the computed aggregates and drafts the natural-language executive summary.

```
// Step 2: Example AI-Generated JSON Plan
{
  "time_column": "Date",
  "time_grain": "week",
  "metrics": [
    {"name": "Total spend", "column": "Price", "op": "sum"},
    {"name": "GPU hours", "column": "Hrs", "op": "sum"}
  ],
  "breakdowns": ["Client Name", "Region"],
  "checks": [
    {"type": "week_over_week", "metric": "Total spend", "by": "Client Name"},
    {"type": "top_n", "metric": "GPU hours", "by": "Region", "n": 5}
  ]
}
```

4. INTERACTIVE CHAT ROUTING

When a user asks, *"Which region spent the most in October?"*:

- The AI does not answer directly.
- It translates the question into a discrete JSON execution plan: `{"metric": "Price", "op": "sum", "by": "Region", "filter": {"Date": "2026-10"}}`.
- Pandas executes the slice.
- The AI is fed the aggregate result to phrase the final response.

> **NVIDIA APPLICATION IMPACT**
>
> This architecture proves you understand enterprise safety constraints. By restricting the AI to a predefined list of operations rather than allowing it to write arbitrary Python code (or un-sandboxed SQL), the application guarantees verifiable, zero-hallucination math. Furthermore, the tool is strictly data-agnostic—it works equally well on GPU telemetry, supply chain logs, or ERP exports.